Linux Process Management: Lab Solutions

Topic 17: Managing Processes • Commands, Descriptions & Explanations

Task 1: Save User Processes to a File

Run the ps command filtered for your current user ($USER) and redirect the output to my_processes.txt:

```
ps -u $USER > my_processes.txt
```

To view the saved contents:

```
cat my_processes.txt
```

Task 2: Launch an Application and Find Its PID

Launch an application in the background (e.g., Spotify, or an alternative like vlc/gedit):

```
spotify &
```

Find the Process ID (PID) using top (locate the process in the list, note its PID, and press 'q' to exit):

```
top
```

Alternatively, retrieve the PID directly using pgrep:

```
pgrep spotify
```

Task 3: Terminate the Process

Kill the process using the PID retrieved in Task 2 (replace <PID> with your process ID, e.g., 1234):

```
kill <PID>
```

If the process does not terminate gracefully, force terminate it:

```
kill -9 <PID>
```

Verify that the process has stopped:

```
pgrep spotify
# Or verify with ps:
ps -p <PID>
```

Task 4: Background and Foreground Job Control

1. Run a long-running command (e.g., ping) in the background using '&':

```
ping google.com > /dev/null &
```

2. View active background jobs:

```
jobs
```

3. Bring the background job back to the foreground:

```
fg %1
```

4. Stop or suspend the job: Press Ctrl + Z to suspend (pause) the process, or Ctrl + C to terminate it completely.

Task 5: Adjust Process Priority (nice and renice)

Start a heavy process with a lower priority using nice (nice values range from -20 [highest priority] to 19 [lowest priority]):

```
nice -n 10 sha256sum /dev/zero &
```

Lower the priority of an already running process using renice:

```
renice +15 -p <PID>
```

> **System Performance Explanation:**
> *Increasing the nice value lowers the CPU scheduling priority of the process, ensuring it yields CPU cycles to interactive and critical tasks so the system remains smooth and responsive.*